HACKATHON SUBMISSION · UNLOCK UNDERUSED EQUIPMENT AND SPACES

CampusShare

A trusted platform that lets a campus share its idle labs, equipment and spaces — safely, fairly, and with a full audit trail.

> **Product document** — problem, users, solution, use cases, design, AI workflow, security and testing
>
> **Team:** shreyvohraa — Shrey Vohra, Pragti Aggarwal
>
> **Code:** github.com/Shrey-Vhr/Code-Conqueror
>
> **Status:** working prototype for one campus (“Northfield University”), 26 automated tests passing

1. Executive summary

Colleges, laboratories, makerspaces and community organisations own valuable equipment and spaces that sit unused for long periods. At the same time, nearby students, researchers, creators and early-stage teams cannot afford to buy or privately rent the same resources. An ordinary booking calendar does not fix this, because sharing institutional property is a **trust problem**: who is allowed to use what, who approves it, what condition it was in, and what happens when something breaks or comes back late.

CampusShare is a working web platform that handles the whole sharing lifecycle with a person in charge of every important decision:

What it does

- Owners list resources with hours, rules, deposits and required training
- Users discover resources and see eligibility before they request
- Requests are ranked by a transparent fairness score
- Owners approve, reject or waitlist, always with a reason
- Check-in and return with condition reports and photos
- Damage, delay and conflict handling with deposit settlement
- An append-only audit log both sides can read

Why it matters

- Idle equipment becomes accessible without owners losing control
- Access is fair and explainable, not first-come or favour-based
- Safety and training are checked before use, not after an accident
- Damage and delays are recorded and handled, not argued about
- AI helps, but never decides access, safety or penalties

The ten-step journey it demonstrates

1 List → 2 Discover & check eligibility → 3 Request → 4 Verify → 5 Review → 6 Approve / reject / waitlist → 7 Check in → 8 Return → 9 Handle damage / delay → 10 Full history

A reviewer can complete this whole journey in the running app in about five minutes, switching between roles with one click.

2. The problem

Who has it

Resource owners
Colleges, university departments, labs, makerspaces and small organisations that hold costly equipment and spaces: laser cutters, 3D printers, cameras, GPU machines, studios, classrooms and workshops.

Would-be users
Students, researchers, creators and early-stage teams nearby who need the same resources but cannot afford to buy them or rent them privately.

How often

Constantly. Specialised equipment is typically in active use only during class or lab hours, so it sits idle in the evenings, at weekends and between projects. Anyone who needs it meets the same obstacles every time: finding out who owns it, whether they are allowed to use it, and who to ask.

What it costs today

Who	Cost
Owners	Equipment idles and loses value. Informal sharing is risky: with no record of who used it, why, and in what condition, owners face misuse, unreported damage, scheduling clashes and unfair allocation. Many simply refuse to share.
Users	Projects stall or never start, because the only options are buying equipment, paying commercial rates, or waiting weeks for access.
Institutions	A plain booking calendar shows availability but does nothing about training requirements, approvals, deposits, damage, late returns or fairness, so these decisions stay manual, unexplained and unrecorded.

Why ordinary booking tools fall short

Need when sharing institutional property	Typical booking calendar
Check training, qualification and standing first	Anyone can book
Owner approval with a recorded reason	Instant, unexplained booking
Fair access when demand exceeds supply	First come, first served; easy to hoard
Condition before and after use	No record

Damage, delay and conflict handling	Handled by email, if at all
A history both sides can trust	None

3. Target users and roles

We start with a university campus, where users already share an institution, a verification path and clear safety training. The same approach extends to makerspaces and community organisations.

Role	Who	Can do in CampusShare
Student / Researcher	People who need access to equipment and spaces	Browse, check eligibility, request, check in and return, cancel or extend, report incidents, submit training evidence
Owner	Lab supervisors and faculty who own resources	List resources, set conditions, review and decide on requests for their own resources
Manager	Department managers	Review requests and resolve incidents for their department, settle deposits and penalties
Administrator	Campus facilities and administration	Verify accounts and certificates, set fairness policy, view analytics and the global audit log

All permissions are enforced on the server, not only hidden in the interface. The signed-in user is re-read from the database on every request, so a suspension or role change applies immediately.

4. The solution

CampusShare turns sharing into one connected workflow. Each stage below maps to a challenge requirement.

Stage	What CampusShare does
List	Owners publish equipment and spaces with location, weekly opening hours, rules, deposit, hourly credit cost, maximum hours per booking, weekly limit and required training.
Discover	Search, filters by type and department, an “I am eligible” toggle, and plain-language AI matching (for example “laser-cut acrylic”).
Qualify	Before requesting, users see a checklist: verified account, good standing, enough credits, department and role rules, and unexpired safety training. Missing items are named.
Request	The form shows duration, cost, refundable deposit and the user's fairness score before they submit. Overlapping times enter a waitlist automatically.
Decide	Owners and managers approve, reject or waitlist, with a mandatory written reason. They see eligibility, conflicts and the score breakdown side by side.

Use	Check-in and return use a checklist, a 1–5 rating, notes and a photo. A worse rating at return drafts a damage incident automatically.
Resolve	Managers settle incidents: no penalty, strike, credit fine or suspension, and refund or forfeit of the held deposit.
Trace	Every approval, rejection, cancellation, extension, check-in, return and incident is written to an append-only audit log.

Booking lifecycle

PENDING or WAITLISTED

- **Request** → `PENDING` (within limits) or `WAITLISTED` (slot already taken)
- `WAITLISTED` → `PENDING` when a slot frees up, by priority score
- `PENDING` → `APPROVED` (owner or manager) or `REJECTED` (with a reason)
- `APPROVED` → `CHECKED_IN` (condition report) or `CANCELLED` (cancel, or no-show with a strike and forfeited deposit)
- `CHECKED_IN` → `COMPLETED` (clean return) or `RETURNED` (damage: incident opened, deposit held) or `OVERDUE` (past end time: delay incident)
- `RETURNED` → `COMPLETED` once the incident is resolved

5. Use cases

Each use case names the actor, the goal, the main flow and the outcome. All are available in the working prototype.

UC1 · List a resource

Actor: Owner

Goal: Make an idle resource available on the owner's terms.

Flow: Open *List New Resource*; enter name, type, category, location, description and safety rules; set weekly opening hours, deposit, hourly credit cost, maximum hours and weekly limit; add requirements (training, role, department, minimum credits); save.

Outcome: The resource appears in the registry with its conditions; the listing is recorded in the audit log.

UC2 · Discover a suitable resource

Actor: Student / Researcher

Goal: Find equipment that fits a need.

Flow: Search and filter, or describe the need in plain language (“high frame rate video capture”) and receive ranked matches with reasons.

Outcome: A short list of fitting resources, with AI matches labelled as a suggestion.

UC3 · Check eligibility and submit training evidence

Actor: Student / Researcher

Goal: Know exactly what is needed before asking, and supply what is missing.

Flow: Open a resource; read the checklist of met and missing requirements; from the profile, submit a safety credential with an evidence note.

Outcome: The certificate shows as pending until an administrator decides.

UC4 · Verify an account or certificate

Actor: Administrator

Goal: Grant access only to qualified people.

Flow: Review pending certificates with an AI pre-screen (advisory); press *Verify* or *Reject*; verify or revoke accounts.

Outcome: The user is notified; verified certificates carry an expiry date and unlock the resource.

UC5 · Request a booking

Actor: Student / Researcher

Goal: Reserve a slot within fair-use limits.

Flow: Choose date, time and purpose; see duration, cost, deposit and fairness score; submit. The system re-checks eligibility, weekly caps, credits and conflicts.

Outcome: The request becomes `PENDING` for review, or `WAITLISTED` with a position and an explained score.

UC6 · Review a request

Actor: Owner / Manager

Goal: Decide with full information and keep control.

Flow: Open the approval queue; compare eligibility snapshot, conflict analysis and priority score; approve or reject with a written reason.

Outcome: The user is told the decision and the reason. Approving a slot moves competing pending requests for the same time to the waitlist, which prevents double booking.

UC7 · Fair waitlist and promotion

Actors: System, Owner

Goal: Give the next most deserving person the slot when it frees up.

Flow: When an approved booking is cancelled, waitlist scores are recalculated with waiting time; the top eligible candidate with no conflict moves to the owner's review queue.

Outcome: The candidate and the owner are notified; the owner still approves.

UC8 · Check in and record initial condition

Actor: Student / Researcher

Goal: Document the state of the equipment before use.

Flow: Open the approved booking; complete the checklist, rate the condition 1–5, add notes and a photo; confirm. (Check-in opens 15 minutes before the start; demo mode lifts this so reviewers can test any time.)

Outcome: The booking is `CHECKED_IN`; the baseline condition report is stored.

UC9 · Return and record final condition

Actor: Student / Researcher

Goal: Close the booking and expose any damage.

Flow: Complete the return checklist, rating, notes and photo; confirm. Usage credits are charged.

Outcome: A clean return completes the booking and refunds the deposit. A rating lower than at check-in drafts a **damage incident** with an AI summary and holds the deposit.

UC10 · Resolve an incident

Actor: Manager

Goal: Handle damage, delay or misuse fairly and on the record.

Flow: Open the incident queue; read the report and the AI summary (suggestion only); write the findings; choose no penalty, a strike, a credit fine, or a suspension; refund or forfeit the deposit.

Outcome: The incident is resolved, the borrower is notified, and the booking completes once every incident on it is resolved.

UC11 · Handle overdue returns and no-shows

Actor: System

Goal: Keep resources moving without manual chasing.

Flow: An automated sweep marks past-due sessions `OVERDUE` and opens a delay incident; cancels no-shows more than 30 minutes late with a strike and forfeited deposit; sends one-hour reminders.

Outcome: The borrower and owner are notified and every step is audited.

UC12 · Extend or cancel a booking

Actors: Student / Researcher, Owner

Goal: Adapt to changing needs without breaking fairness.

Flow: Request an extension (checked against conflicts, per-booking maximum and weekly caps) or cancel (deposit refunded, waitlist promoted).

Outcome: The affected people are notified and the change is logged.

UC13 · Govern the system

Actor: Administrator

Goal: Keep rules fair and see how the system is used.

Flow: Edit fairness policy (weekly cap, score weights, monthly credit grant); review usage statistics and fair-use flags; read the global audit ledger.

Outcome: Every policy change is itself logged with who changed it and why.

UC14 · See the full history

Actors: Student, Owner, Manager, Administrator

Goal: Trust the record.

Flow: Open any booking or incident to see the timeline of decisions, reasons, condition reports and incidents, with who did each step and when.

Outcome: Both parties share one complete, append-only history of the transaction.

6. Fair allocation

When demand exceeds availability, requests are ranked by a score that is shown to both the user and the reviewer. The score **ranks** requests; a person still approves or rejects.

```
Priority = 100
  − 2.5 × hours used in the last 30 days
  + waitlist time bonus (up to +30)
  + purpose bonus (Research +25 · Coursework +20 · Club +5)
  + role bonus (Researcher +15 · Student +10)
  − 25 × conduct strikes
```

Measure	How it prevents hoarding or unfairness
Heavy-use penalty	People who have used a lot recently rank lower than people who have not
Waitlist time bonus	Waiting counts, so nobody is starved
Weekly hour cap	Campus-wide limit per person (default 20 hours) plus per-resource weekly limits
Credits and deposits	Monthly credit grant, hourly credit costs, refundable deposit held until a clean return
Strikes	No-shows and conduct penalties lower priority; three strikes block booking
Explainability	The full score breakdown is visible to the user and the reviewer, and saved with the booking
Admin control	Weights and caps are editable by an administrator; every change is logged

7. AI, with a person in charge

AI is used where it saves time and never decides access, safety or penalties.

Used for

- Matching a plain-language need to resources
- Pre-screening certificate evidence
- Summarising incidents and suggesting a severity
- Explaining unusual usage patterns to an administrator

Never used for

- Approving or rejecting a booking
- Charging credits or deposits
- Assigning strikes or penalties
- Suspending anyone

Prompt architecture and data flow

1. A user or staff member acts in the interface; a server action first checks their login and role.
2. The server reads the needed records and builds a prompt with a fixed role and task, the case data, and the exact output format (JSON for matching, certificate review and incident summaries). Usage prompts carry no names, only department and counts.
3. One call is made to the OpenAI API (model `gpt-4o-mini`, low temperature, 4-second timeout).
4. The output is validated in code: resource ids must exist, severity must be an allowed value, malformed output is discarded.
5. If there is no key, the call fails or times out, or validation fails, a rule-based fallback returns a result (keyword scoring, templated summaries), so the app never breaks.
6. Results are stored so they are not requested twice, and shown with the label **"AI suggestion — requires human review"**.
7. A person makes the decision, and that decision is written to the audit log.

There are no agents or multi-step chains in the product: each feature is a single prompt and call. AI output never changes a booking status, balance, strike or account. For example, the official severity of an auto-drafted damage incident comes from a fixed rule; the AI's suggested severity is stored separately for the manager to consider.

8. Security, privacy and accountability

Area	Controls
Access control	Server-side role and ownership checks on every action; the user is re-read from the database on each request
Credentials	bcrypt-hashed passwords; signed, HTTP-only session cookies; secrets required in production
Input safety	All inputs validated with Zod; allowed values enforced for types, statuses and severities
Least information	Users see their own bookings; owners and managers see only what they need to review; certificate documents are visible to administrators; no personal names are sent to the AI for usage analysis
Guarded endpoints	The overdue-sweep endpoint needs a secret; demo logins can be switched off with a setting
Audit log	Append-only; written in the same database transaction as each change; no update or delete path in the application
Human oversight	Access, safety and penalty decisions are always made by an authorised person, with a recorded reason

9. Architecture and technology

Stack
Next.js 15 (App Router, Server Actions) · React 19 · TypeScript · Tailwind CSS 4 · Prisma 6 with SQLite · Zod 4 · jose · bcryptjs · OpenAI SDK (optional) · Vitest

Core data
Users, Resources and requirements, Trainings and certifications, Bookings, Condition reports, Incidents, Notifications, Audit log, Fairness settings

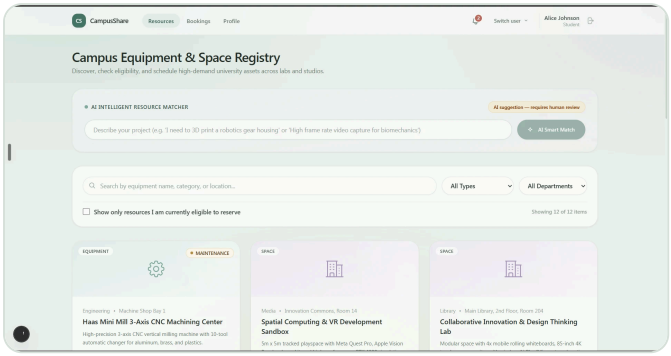
Module	Responsibility
eligibility	Checks account, standing, credits, role, department and unexpired training; returns what is met and what is missing
conflicts	Detects overlaps, opening-hours violations, maintenance and maximum-duration breaches
fairness	Computes the priority score; enforces weekly caps; grants monthly credits; reads the admin policy
bookingService	The booking state machine: request, approve, reject, cancel, check in, return, extend, waitlist promotion
incidentService	Resolves incidents, applies penalties, settles deposits, completes bookings
overdue	Marks overdue sessions, cancels no-shows, sends reminders, refreshes waitlist scores
notifications / audit	In-app alerts and the append-only audit trail
ai	Prompt building, validation and rule-based fallbacks for the four AI features

Quality assurance

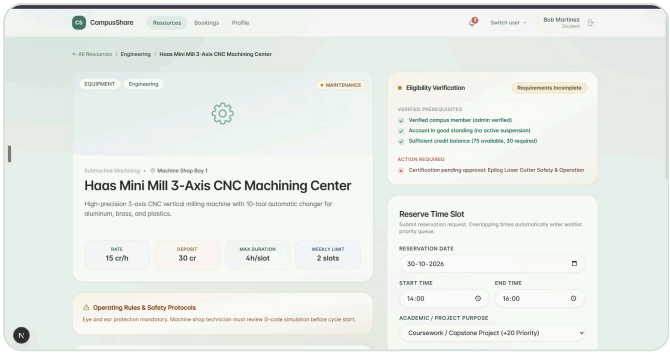
26 automated tests (unit and integration) run against a temporary copy of the database. They cover eligibility, fairness scoring, conflict detection, booking state, and integration tests on a real database for the full lifecycle, double-booking prevention, waitlist promotion, overdue and no-show sweeps, incident resolution with deposit settlement, and role boundaries. The project also builds and type-checks cleanly.

10. The product

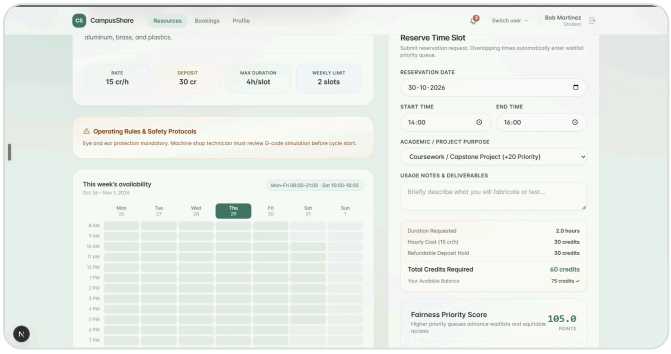
Screens from the running prototype, in lifecycle order.



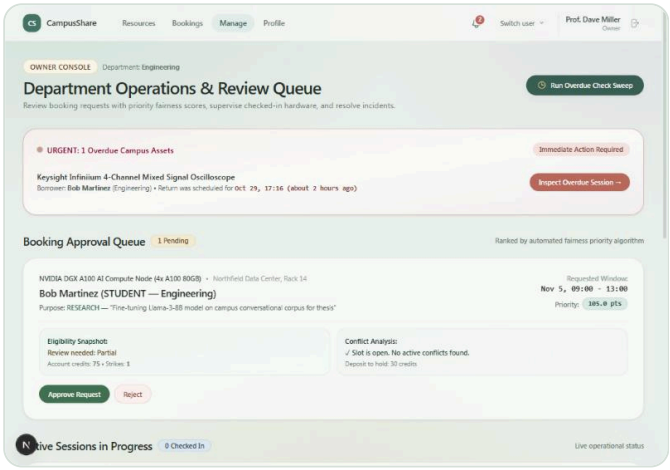
Discover. Search, filters and plain-language AI matching, labelled as a suggestion.



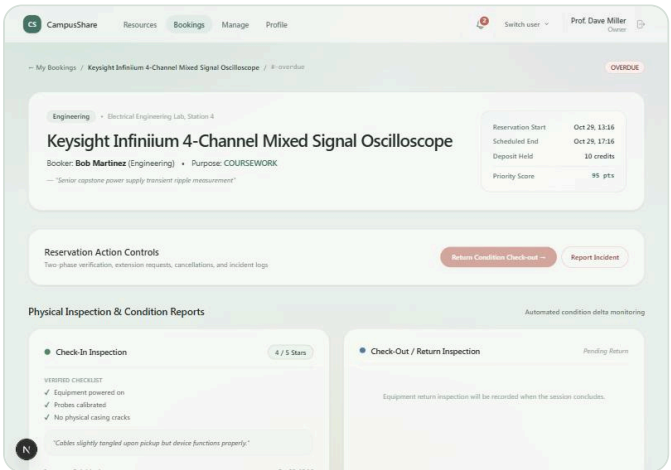
Qualify. What the user meets and what is still missing, next to the booking form.



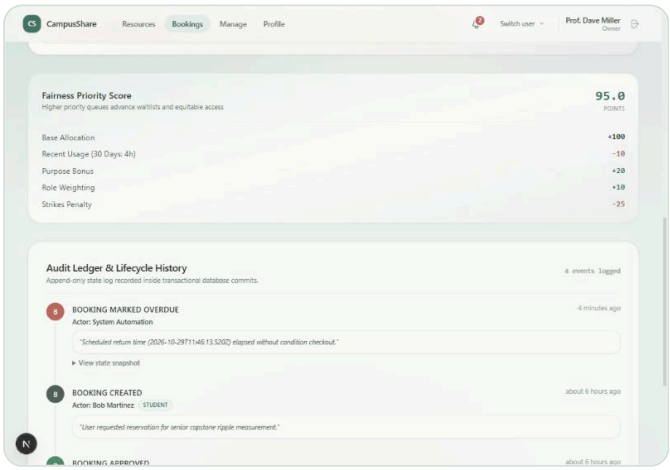
Plan. Weekly availability, cost and deposit, and the fairness score.



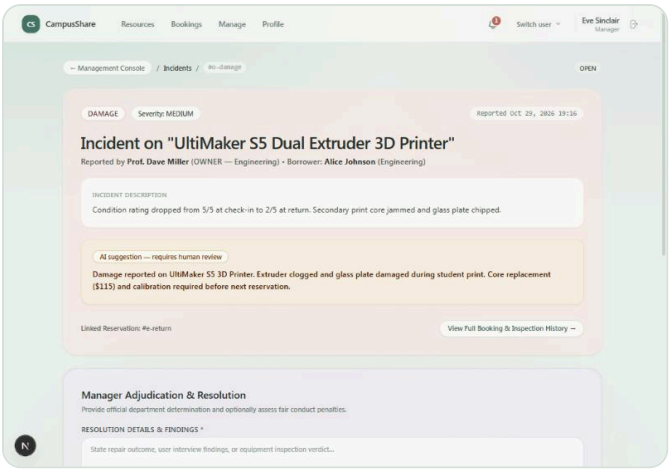
Decide. The owner's queue with eligibility, conflicts and score.



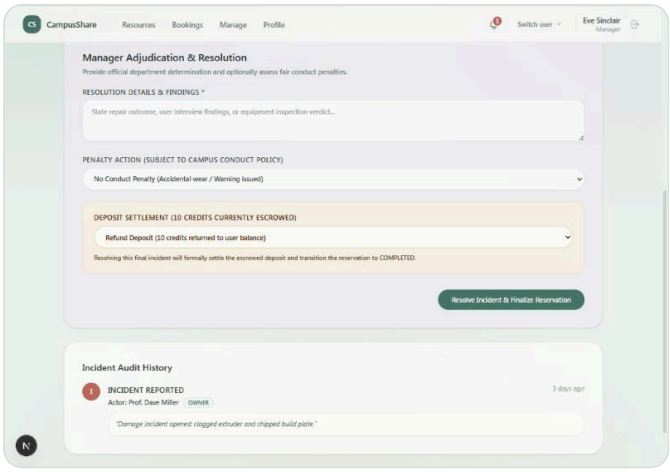
Record. Check-in condition report with the return inspection pending.



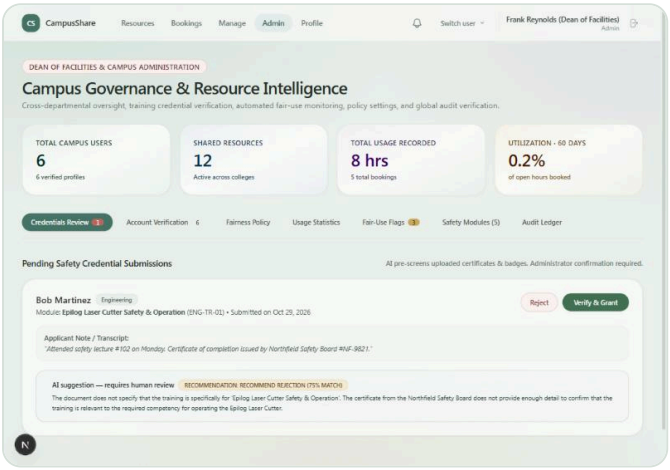
Explain. Score breakdown and the append-only audit ledger.



Report. Damage incident with an AI summary that is only a suggestion.



Resolve. Penalty options and deposit refund or forfeit.



Verify. AI advises on a certificate; the administrator decides.

11. How it meets the challenge brief

Expected outcome	In CampusShare
Listings with availability, location, access conditions, rules	Owner listing form; resource page with weekly availability calendar
Search, filtering or intelligent matching	Filters, eligibility toggle, AI matching
Profiles and verification	Profile page; administrator verification of accounts and certificates
Eligibility checks	Pre-request checklist covering training, standing, credits, role, department
Approval, rejection and waitlist	Reviewer decisions with mandatory reasons; automatic waitlist
Transparent handling of competing requests	Conflict detection; explained priority score; double-booking prevention

Check-in and check-out; before/after condition	Checklist, rating, notes and photo at both points
Damage, delay and incident reporting	Incident queue; automatic damage drafts; overdue delay incidents
Notifications	Approvals, rejections, waitlist moves, reminders, cancellations, overdue
Fair-usage measures	Caps, credits, deposits, priority score, no-show strikes
Role-based access	Student, Researcher, Owner, Manager, Administrator
Auditable history	Append-only audit log, visible in booking, incident and admin views
Privacy and security	See section 8
AI where valuable, humans decide	Section 7; every AI result is reviewable and labelled

12. What makes it distinctive

- **Trust first.** Eligibility is checked before a request, and users see exactly what is missing, so they are not rejected afterwards without explanation.
- **Explainable fairness.** A transparent score, caps, credits and deposits; every user can see why they were approved, rejected or waitlisted.
- **One connected lifecycle.** Request, approval, condition records, return, incident and deposit settlement live in one flow. A worse condition at return drafts an incident automatically, so accountability does not depend on someone remembering to report it.
- **A shared record.** An append-only history both owner and user can read.
- **AI as advisor.** Useful, labelled and optional; the product works fully without it.

13. Limitations and future work

This is a prototype for a single campus.

- Notifications are in-app only; email and SMS are future work.
- Certificate evidence is a text note; document upload and verification are future work. Condition photos are stored in the database; a real deployment would use a file store.
- Sign-in is simulated with demo accounts; a real deployment would use campus single sign-on.
- One institution and one campus time zone; multi-institution sharing is a natural extension.
- SQLite suits a demo; production would use a hosted database.

14. Try it

Repository: github.com/Shrey-Vhr/Code-Conqueror (the README has a two-minute setup and a five-minute guided tour).

Run: `npm install`, `npm run db:reset`, `npm run dev`, then open `localhost:3000` and pick a demo account.

Role	Email	Password
Student	alice@northfield.edu	Alice2024!
Student	bob@northfield.edu	Bob2024!
Researcher	carol@northfield.edu	Carol2024!
Owner	dave@northfield.edu	Dave2024!
Manager	eve@northfield.edu	Eve2024!
Administrator	frank@northfield.edu	Frank2024!

Demo credentials for the seeded sample campus only.

15. Team and development approach

Team shreyvohraa

- **Shrey Vohra** — research, product design, UI/UX, AI workflows and systems, deployment
- **Pragti Aggarwal** — backend and frontend development with AI agents, documentation, deployment

The product was researched and specified, then built with AI coding assistance (Claude and Antigravity), followed by several rounds of independent review and testing to catch security and logic problems before submission.